



上海科技大学
ShanghaiTech University

From Chaofan Li

【GeekHour】一小时Git教程 ->



立志成才报国 裕民



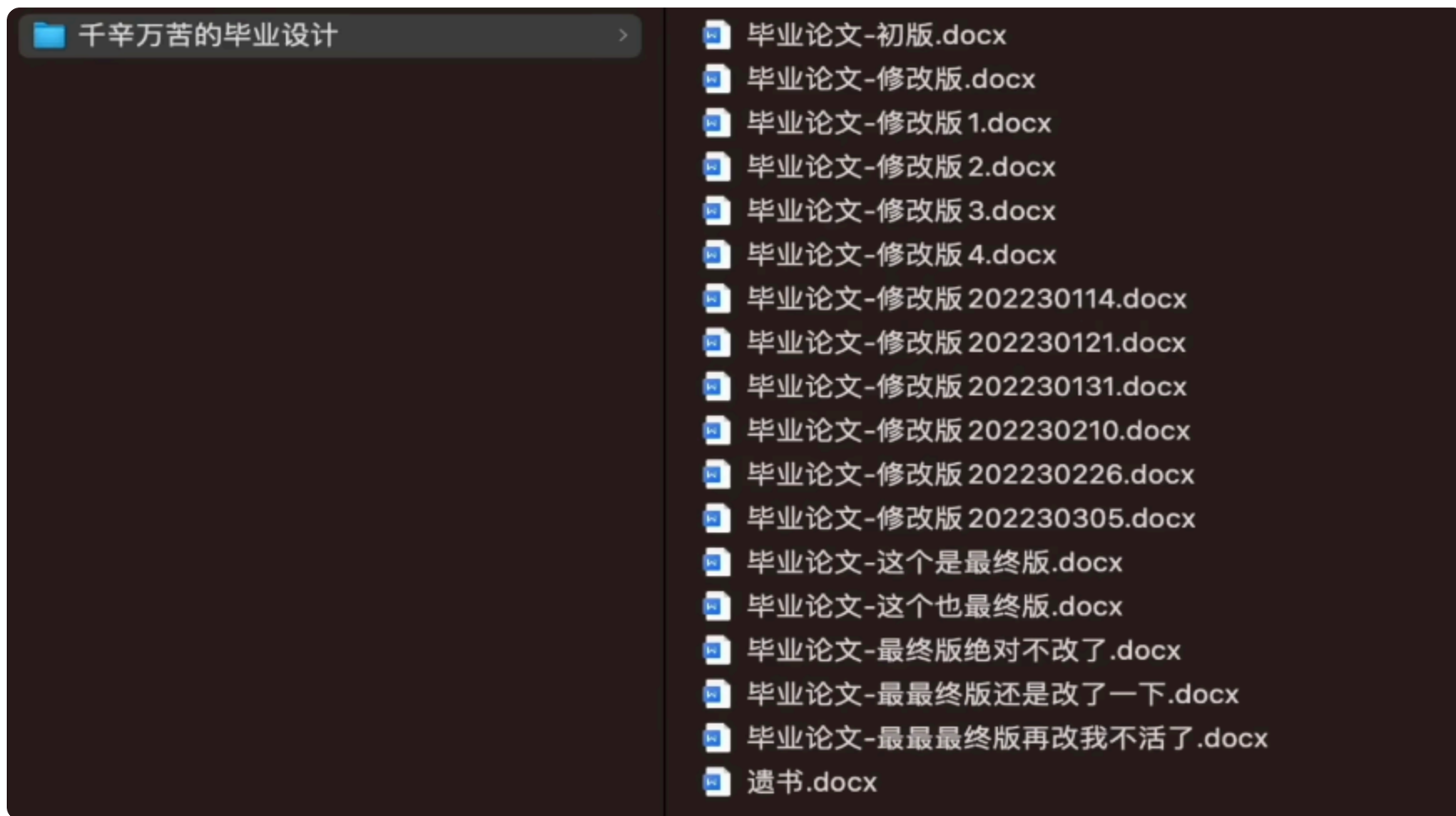
上海科技大学
ShanghaiTech University

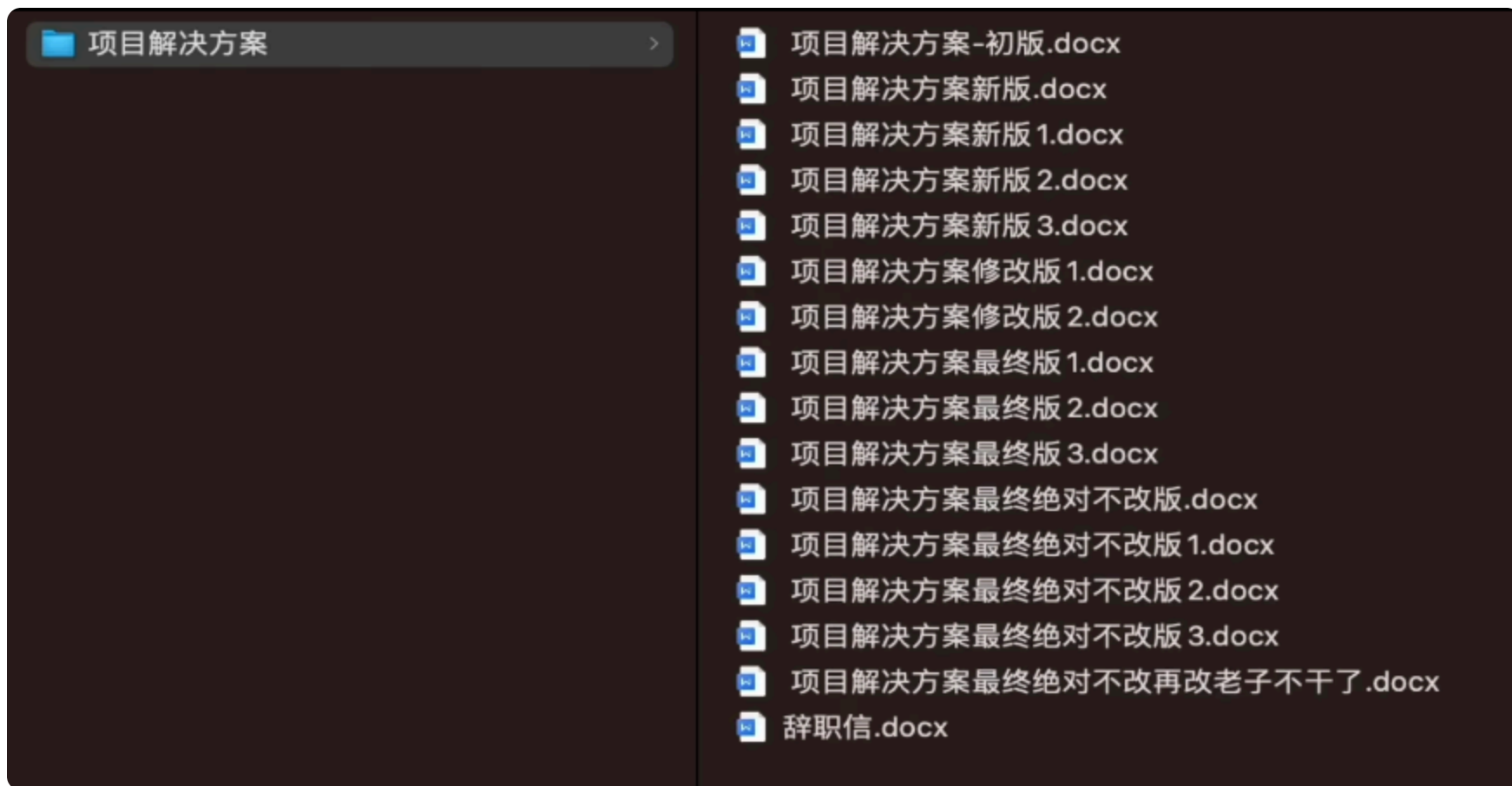
- 免费
- 开源
- 分布式管理版本控制系统



立志成才报国 裕民









icy / SI100B_2024Spring 0

Search Type [Z] to s

Issues Pull requests Actions Projects Wiki Security Insights Settings

Commits

main	All users	All time
Commits on May 17, 2024		
1 Heaticy committed on May 17	f8228cc	
tutorial2 snkrssubscriberdqh committed on May 17	3819d7e	
PJ Presentation HypoxanthineOvO committed on May 17	8c4418d	
tutorial2 snkrssubscriberdqh committed on May 17	d609d98	
tutorial2 snkrssubscriberdqh committed on May 17	2607fc2	
tutorial2 snkrssubscriberdqh committed on May 17	c76eb21	
Commits on May 16, 2024		
Merge branch 'main' of github.com:Heaticy/SI100B_2024Spring Heaticy committed on May 16	1a268ef	
moss Heaticy committed on May 16	2d8c1eb	
Commits on May 15, 2024		
Merge branch 'main' of https://github.com/Heaticy/SI100B_2024Spring HypoxanthineOvO committed on May 15	03a72dd	
Project Presentation HypoxanthineOvO committed on May 15	357d4f0	
Complete Reference Hypoxanthine authored and Hypoxanthine committed on May 15	83d4d81	



- 有一天小李和小刘CS100大作业组队，他们要写一个植物大战僵尸的游戏。
- 他们约好一个人写植物，一个人写僵尸，写完自己的part就用微信把plant.c文件和zombie.c文件用微信传过去看看能不能work
- 小李下载了小刘写的zombie.c，然后发现不能work，然后改了自己写的plant.c然后微信传给小刘
- 小刘下载了小李的plant.c以后又发现自己少写了个僵尸，补上以后正准备给小李时候收到了来自他的plant.c，然后替换了plants.c到最新版本以后发现又不能和自己新写的一起work了，于是他把自已zombie.c发过去让小李改好自己的plants.....





- 此时小李已经感到有些生理不适了，但他还是按着最新的zombie.c改好了自己的plant.c并微信发过去
- 但是小刘是个完美主义者，他一定要把自己的僵尸弄好看点于是又弄了一版zombie.c然后发给lcf让他提交
- 提交时候小李手滑没弄错了版本，把初代zombie.c用zip压缩后提交了
- 当然这个zip编译都编译不了的，两人天神下凡怒砍一个B，小刘对这次合作非常生气，删掉了小李的好友

(以上故事纯属虚构)





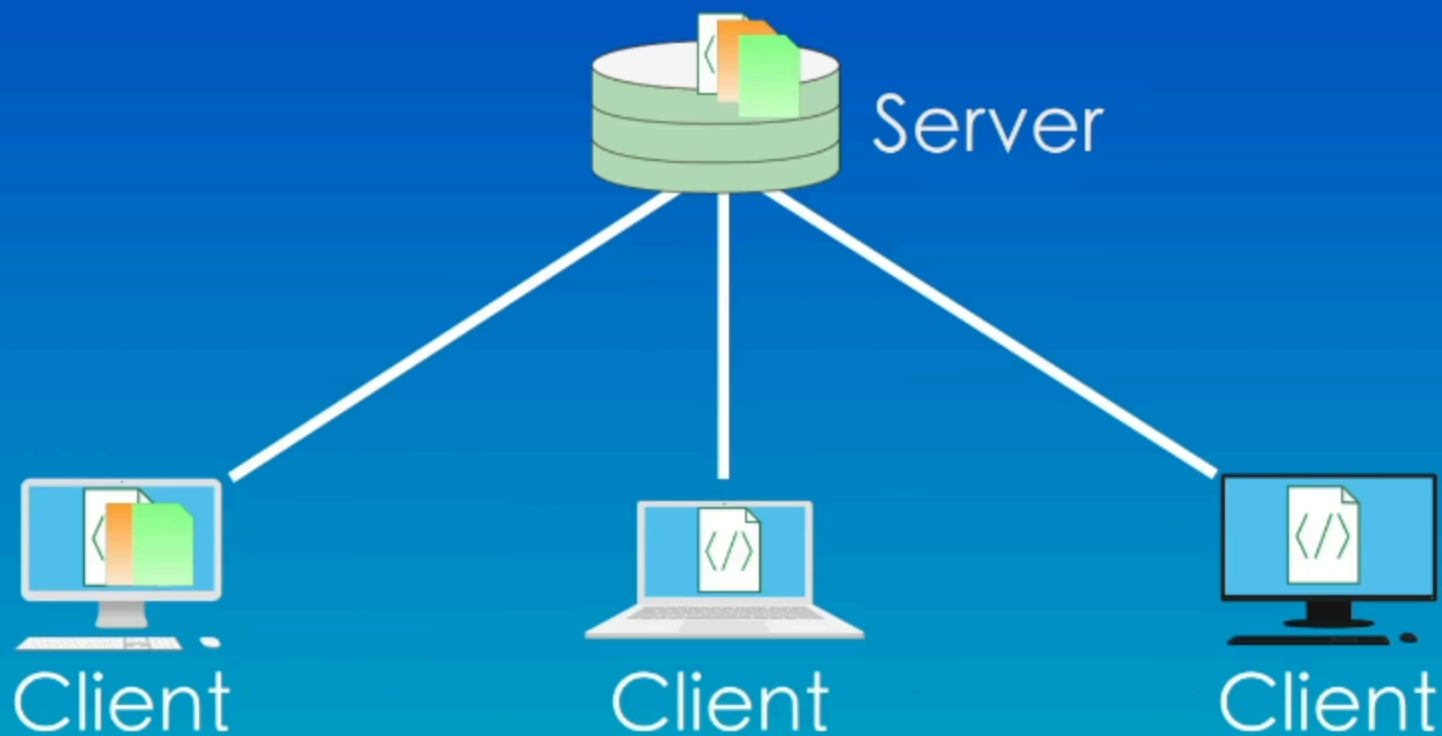
上海科技大学
ShanghaiTech University



立志成才报国 裕民



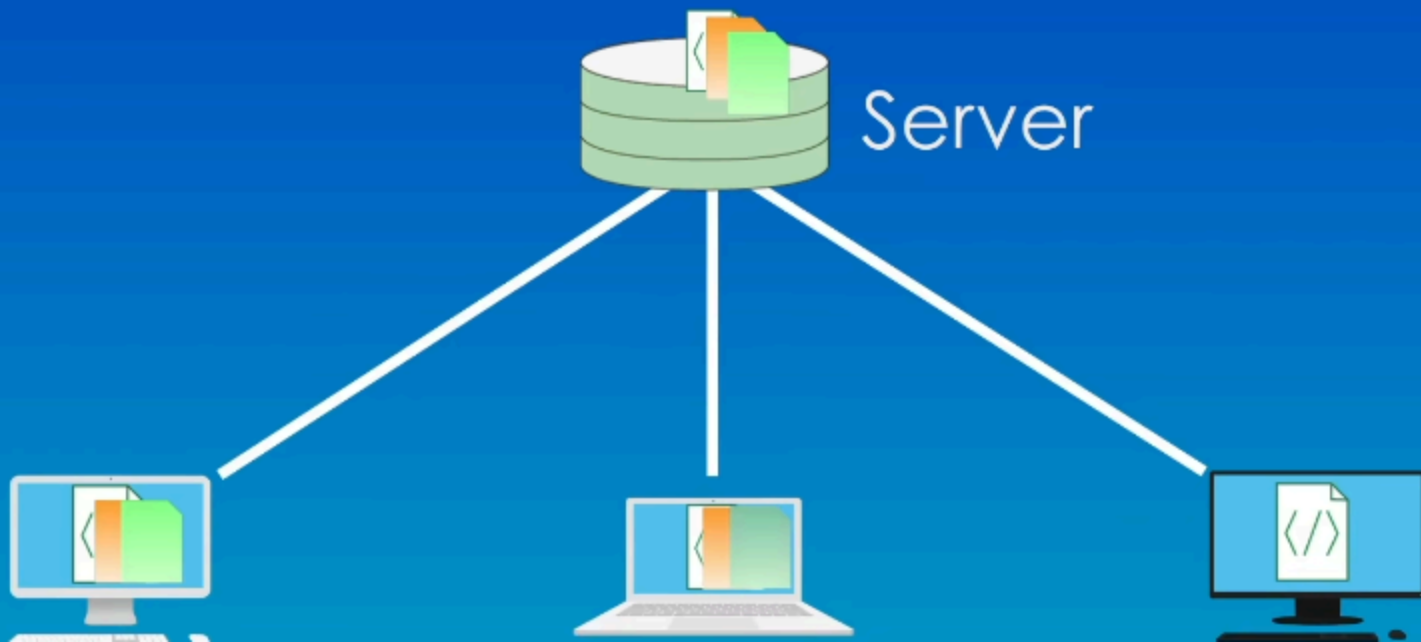
集中式





编辑之前从云端下载，编辑之后实时上传：需要时刻保持和云端的连接

集中式





分布式



当需要将我们的修改内容分享给其他人的时候



分布式





分布式





分布式



因为免费开源 速度快 功能强大



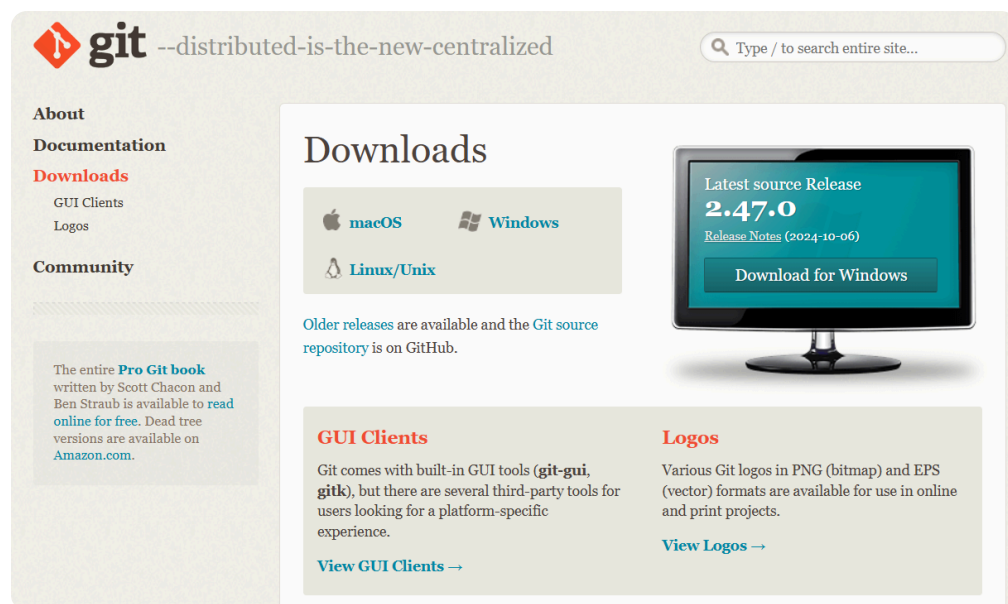
分布式



因为免费开源 速度快 功能强大



git下载连接 ->



用终端里面的 `git -v` 去检测是否安装好



为了识别是谁编辑上传的文件，需要绑定好git的用户名和邮箱（和后面的github用户名邮箱一致）

```
git config --global user.name "Heaticy"  
git config --global user.email "1712371381@qq.com"
```

邮箱的双引号可以省略

这两个命令在一台电脑上只需要执行一次即可





仓库：把你当前工作区的文件目录设置为一个仓库，git会记录你整个目录下的文件版本改动

- 方式一：git init：从本地创建
- 方式二：git clone：从云端复制现有的仓库





工作区：资源管理器里面看到的文件夹





工作区

Working Directory

暂存区

Staging Area/Index



本地仓库

Local Repository

暂存区：写了一部分代码，临时保存，可以随时轻易回退一个版本的临时区



本地仓库：记录了所有文件的版本改动，和版本信息

工作区

Working Directory

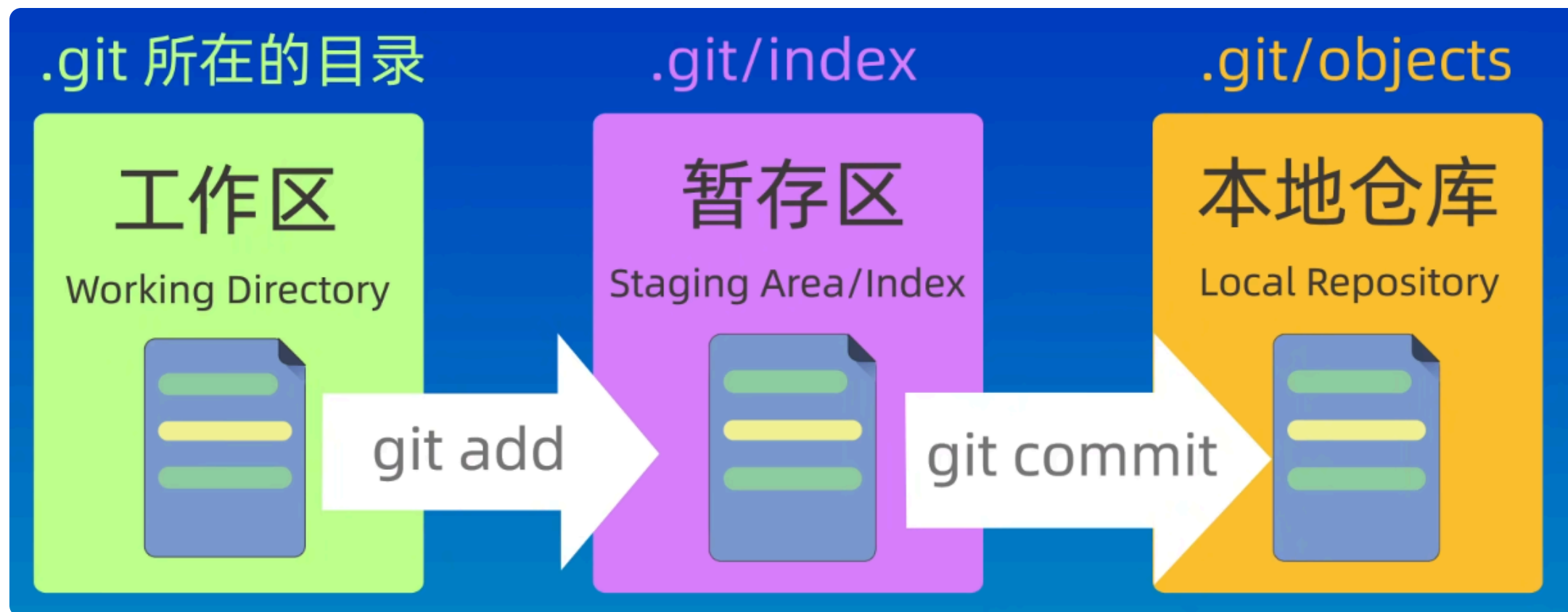
暂存区

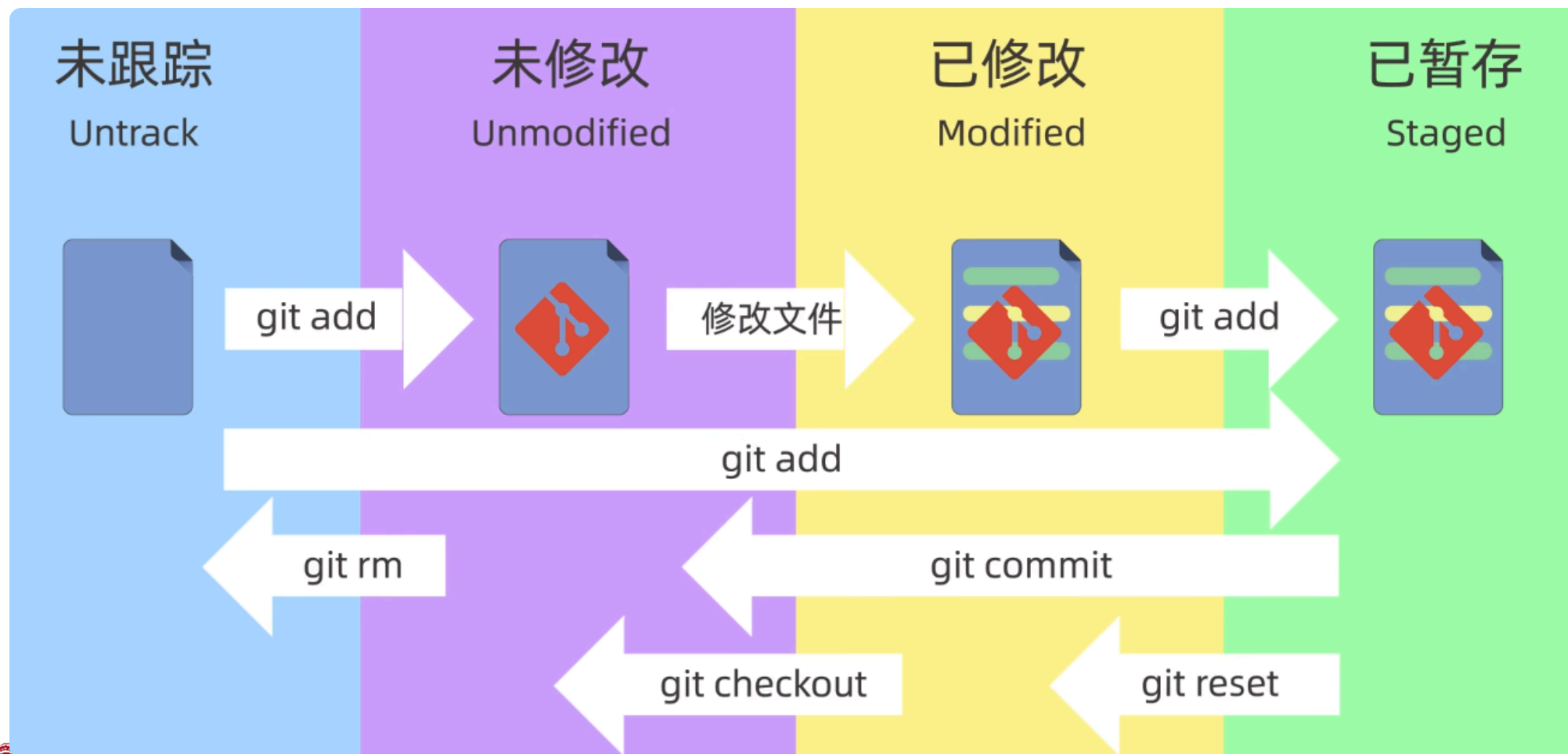
Staging Area/Index

本地仓库

Local Repository









```
PS E:\gitpre> git init  
Initialized empty Git repository in E:/gitpre/.git/
```





我们先创建一个a.c文件，然后把他加入暂存区

```
PS E:\gitpre> git add a.c
PS E:\gitpre> git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
    new file:   a.c
```

如果`git add`后面加`.`则是把所有工作区所有的文件加入暂存区，`.`表示所有的文件

```
PS E:\gitpre> git add .
```



然后上传至本地仓库，`-m`后的引号内容为提交信息

```
PS E:\gitpre> git commit -m "first commit"
[master (root-commit) bf9079d] first commit
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 a.c
```

当然不加`-m`在**linux**, **macOS**上也可以，则是使用vim等编辑器来添加提交信息

```
git commit -a -m "first commit"
```

可以把已经追踪的文件加入暂存并提交，但是得先add





1b793dbe4532f70cb1b9bec6005354b49ebbba2e 是提交ID

```
PS E:\gitpre> git log  
commit 1b793dbe4532f70cb1b9bec6005354b49ebbba2e (HEAD -> master)  
Author: Heaticy <1712371381@qq.com>  
Date: Mon Oct 14 18:39:03 2024 +0800
```

fist commit

用git oneline可以获得更简洁的信息

```
PS E:\gitpre> git log --oneline  
1b793db (HEAD -> master) fist commit
```



git status

查看仓库的状态

git add

添加到暂存区

可以使用通配符，例如：git add *.txt
也可以使用目录，例如：git add .

git commit

提交

只提交暂存区中的内容，不会提交工作区中的内容

git log

查看仓库提交历史记录

可以使用 --oneline 参数来查看简洁的提交记录





- `git reset --soft` 回退到某个版本，**保留**工作区和暂存区
- `git reset --hard` 回退到某个版本，**丢弃**工作区和暂存区
- `git reset --mixed` (**默认**) 回退到某个版本，**保留**工作区，**丢弃**暂存区





创建一个新文件叫1.txt



然后提交

```
PS E:\gitpre> git commit -m "add 1.txt"
[master 8665890] add 1.txt
1 file changed, 1 insertion(+)
create mode 100644 1.txt
```



```
PS E:\gitpre> git log
commit b0c1dee188f1659236f57df625391f4b07de5dbc (HEAD -> master)
Author: Heaticy <1712371381@qq.com>
Date:   Mon Oct 14 18:54:56 2024 +0800
```

```
    add 1.txt
```

```
commit 1b793dbe4532f70cb1b9bec6005354b49ebbba2e
Author: Heaticy <1712371381@qq.com>
Date:   Mon Oct 14 18:39:03 2024 +0800
```

```
    fist commit
```





```
PS E:\gitpre> git reset --soft 1b793db
```

```
PS E:\gitpre> ls #查看工作区，工作区没变
```

```
目录: E:\gitpre
```

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a----	2024/10/14 18:52	3	1.txt
-a----	2024/10/14 18:26	0	a.c

```
PS E:\gitpre> git ls-files #查看暂存区，暂存区没变
```

```
1.txt
```

```
a.c
```

```
PS E:\gitpre> git status
```

```
On branch master
```

```
Changes to be committed:
```

```
(use "git restore --staged <file>..." to unstage)
```

```
new file: 1.txt
```





```
PS E:\gitpre> git reset --mixed 1b793db
```

```
PS E:\gitpre> ls      #查看工作区，工作区没变
```

```
    目录: E:\gitpre
```

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a----	2024/10/14 18:52	3	1.txt
-a----	2024/10/14 18:26	0	a.c

```
PS E:\gitpre> git ls-files  #查看暂存区，暂存区被清空了
```

```
a.c
```

```
PS E:\gitpre> git status
```

```
On branch master
```

```
Untracked files:
```

```
    (use "git add <file>..." to include in what will be committed)
```

```
    1.txt
```

```
nothing added to commit but untracked files present (use "git add" to track)
```





```
PS E:\gitpre> git reset --hard 1b793db  
HEAD is now at 1b793db fist commit  
PS E:\gitpre> ls
```

目录: E:\gitpre

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a----	2024/10/14 18:26	0	a.c

```
PS E:\gitpre> git ls-files  
a.c
```



```
PS E:\gitpre> git reset --hard 1b793db
HEAD is now at 1b793db fist commit
PS E:\gitpre> git log
commit 1b793dbe4532f70cb1b9bec6005354b49ebbba2e (HEAD -> master)
Author: Heaticy <1712371381@qq.com>
Date:   Mon Oct 14 18:39:03 2024 +0800

    fist commit
```

用了hard以后git log都没后面版本的记录了

但你若是还记得前面版本号的话还是可以回去的

```
PS E:\gitpre> git reset --hard b0c1dee188f1659236f57df625391f4b07de5dbc
HEAD is now at b0c1dee add 1.txt
```



- 提交之前，可以看看自己这一版改了啥

查看工作区、暂存区、本地仓库之间的差异

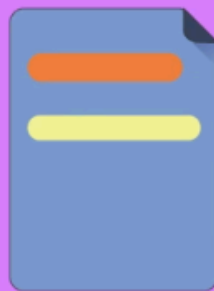
工作区

Working Directory



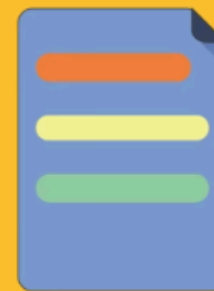
暂存区

Staging Area/Index



本地仓库

Local Repository



git diff





- 提交之前，可以看看自己这一版改了啥

```
PS E:\gitpre> git diff
diff --git a/1.txt b/1.txt
index 9d07aa0..183adcd 100644
--- a/1.txt
+++ b/1.txt
@@ -1,2 @@
-111
\ No newline at end of file
+111^M
+222
\ No newline at end of file
```





```
PS E:\gitpre> git diff b0c1dee188 1b793db
diff --git a/1.txt b/1.txt
deleted file mode 100644
index 9d07aa0..0000000
--- a/1.txt
+++ /dev/null
@@ -1 +0,0 @@
-111
\ No newline at end of file
```





HEAD~表示上一个版本, HEAD表示当前版本
HEAD~2表示前两个版本

```
PS E:\gitpre> git diff HEAD~ HEAD
diff --git a/1.txt b/1.txt
new file mode 100644
index 0000000..9d07aa0
--- /dev/null
+++ b/1.txt
@@ -0,0 +1 @@
+111
\ No newline at end of file
```





第三个参数填文件名也可以

```
PS E:\gitpre> git diff HEAD~ HEAD 1.txt
diff --git a/1.txt b/1.txt
new file mode 100644
index 0000000..9d07aa0
--- /dev/null
+++ b/1.txt
@@ -0,0 +1 @@
+111
\ No newline at end of file
```





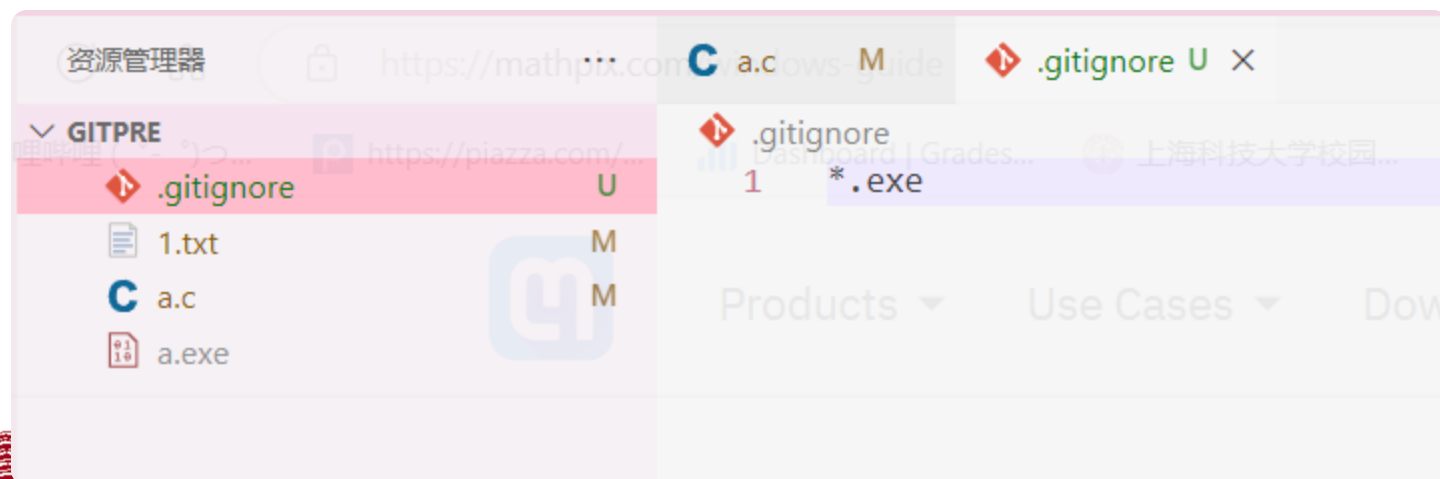
git应该忽略哪些文件?

- 系统或者软件自动生成的文件
- 编译产生的中间文件和结果文件
- 运行时生成日志文件、缓存文件、临时文件
- 涉及身份、密码、口令、秘钥等敏感信息文件
- .o .class .env .vscode .DSstore .zip





```
C a.c M X
C a.c > main()
1  #include <stdio.h>
2
3  int main()
4  {
5      printf("Hello, World!\n");
6      return 0;
7  }
```





上海科技大学
ShanghaiTech University

github是个存代码和仓库的网站，如果多人协作，则需要把本地仓库上传至云端(github,gitee等)



立志成才报国 裕民



ssh是一个远程连接协议，可以让你的git和github云端之间连接，从而可以从本地上传github

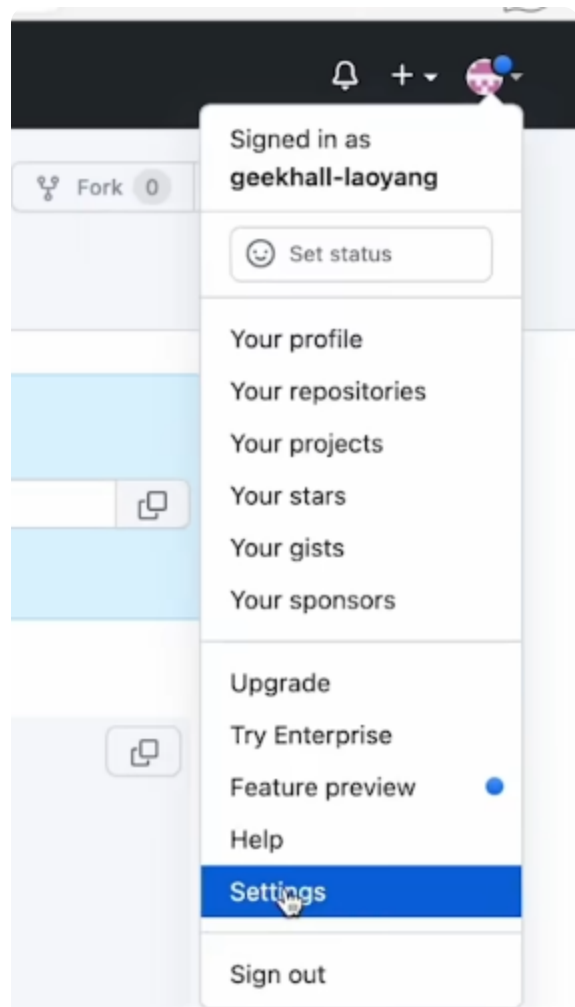
```
ssh-keygen
```

然后一直回车

```
Your identification has been saved in test
Your public key has been saved in test.pub
The key fingerprint is:
SHA256:JhqxdhNuEaUZgmA4eayTFyD6ZwgGMpjnYki0JDXJsa8 yiny@Ares
The key's randomart image is:
+---[RSA 4096]-----+
|XX++. o..          |
|&+0o . =           |
|X0... =            |
|0=oo + o           |
|. +o B * S         |
|  = = +            |
| E .               |
+---[SHA256]-----+
```



上海科技大学
ShanghaiTech University



立志成才报国 裕民





Jasper Yang
Your personal account

Public profile

Account

Appearance

Accessibility

Notifications

Access

Billing and plans

Emails

Password and authentication

Sessions

SSH and GPG keys

Organizations

Moderation

Code, planning, and automation

Repositories

Codespaces

Packages

Copilot

Go to your profile

Public profile

Name

Jasper Yang

Your name may appear around GitHub where you contribute or are mentioned. You can remove it at any time.

Public email

Select a verified email to display

You have set your email address to private. To toggle email privacy, go to [email settings](#) and uncheck "Keep my email address private."

Bio

Tell us a little bit about yourself

You can @mention other users and organizations to link to them.

Pronouns

Don't specify

URL

geekhall.cn

Profile picture



Edit

在页面左侧有一个SSH和GPG密钥配置的选项

才报国 裕民



Jasper Yang

Your personal account

Go to your profile

Public profile

Account

Appearance

Accessibility

Notifications

Access

Billing and plans

Emails

Password and authentication

Sessions

SSH and GPG keys

Organizations

Moderation

Code, planning, and automation

Repositories

Codespaces

Packages

SSH keys

New SSH key

There are no SSH keys associated with your account.

Check out our guide to [generating SSH keys](#) or troubleshoot [common SSH problems](#).

GPG keys

New GPG key

There are no GPG keys associated with your account.

Learn how to [generate a GPG key and add it to your account](#).

Vigilant mode

☐ Flag unsigned commits as unverified

This will include any commit attributed to your account but not signed with your GPG or S/MIME key.

Note that this will include your existing unsigned commits.

[Learn about vigilant mode.](#)



The screenshot shows the GitHub account settings page for 'Jasper Yang'. The left sidebar contains navigation links: Public profile, Account, Appearance, Accessibility, Notifications, Access, Billing and plans, Emails, Password and authentication, Sessions, SSH and GPG keys (highlighted), Organizations, and Moderation. The main content area is titled 'SSH keys / Add new'. It includes a 'Title' field with the value 'test', a 'Key type' dropdown set to 'Authentication Key', and a 'Key' text area containing a long SSH key. Below the key text area is a green button labeled 'Add SSH key'.

SSH keys / Add new

Title: test

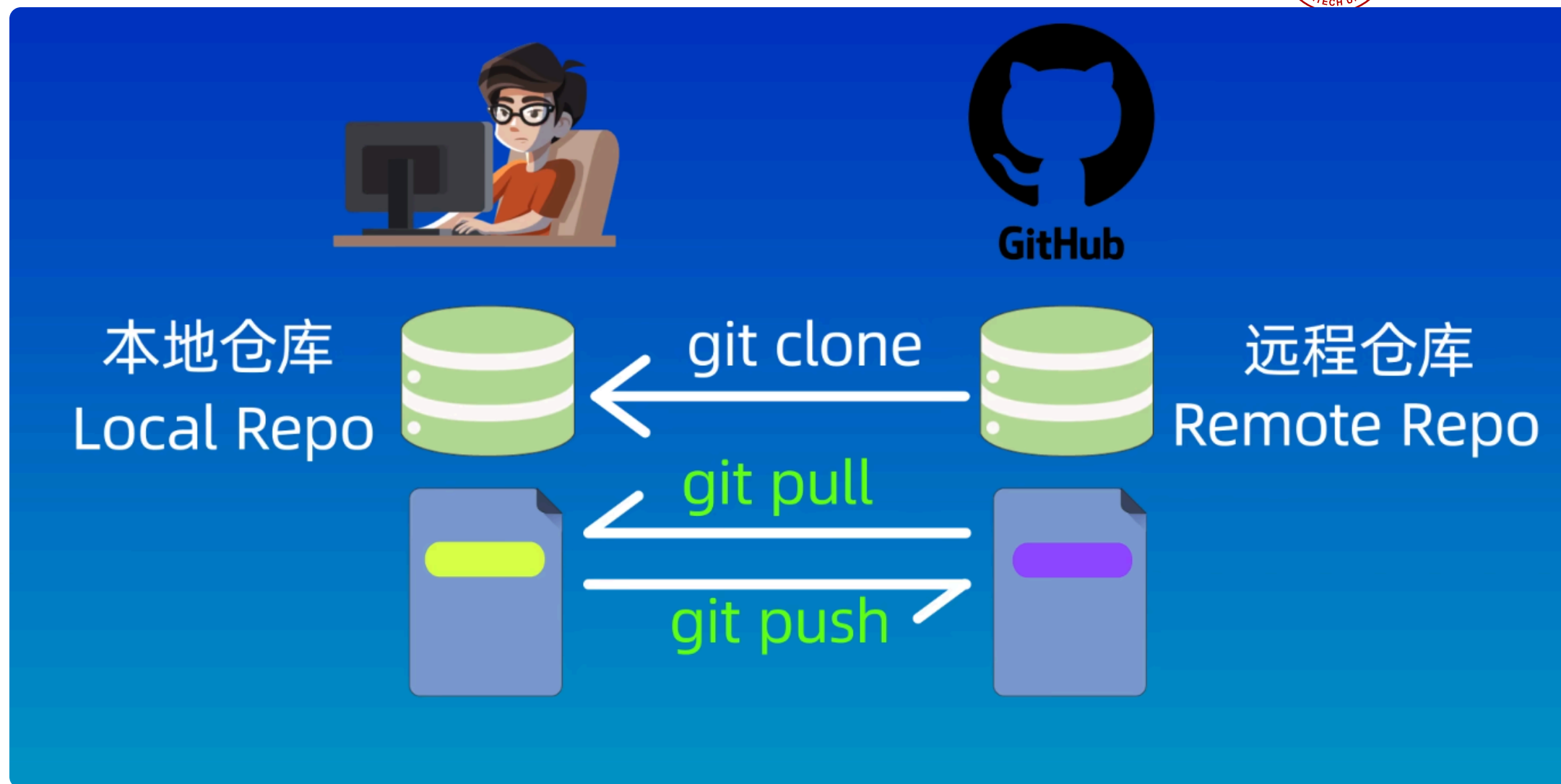
Key type: Authentication Key

Key: ssh-rsa AAAAB3NzaC1yc2EAAAADAQABAAQCAQIDW9nrITxT0oKjH6SR4x+AYPANqn0IgNYF+yRWtlljiREXXDAKT+ei/MaW1OGYC4eyLXcwHPLegjRv9XCnNyoozzV3REHLu3oFwZ+5pDg2e1ZOirgBP24eilAfXi4Ktz9IfODVQ30xmes9TwOxWY3nVXqq7vvllLhWZC+n7hPg0gGZnVRF0AfgoEJRFXeRyqSFQ4WEj+TDeID6UZhjhZNQwdcUsAv/vgpxZMOiJh00Xato6mc/Q78BBOft8C12YNEJPMCW3hO9t1y2L69qti5Jac5QDyXFaseY659Slrgpj03LZ9AKvurkWIC10q6gLBHqBB0kHNKeqOuVUHW7CxR6Kqk8GgK8sdn3gkoBkEqIGzYTCOEK15cGxOfgevs4QkhMOZcSZk7C7QxkVnDteYg91R//MAennjblKehZg/Gj65W82Eg+rP3PS2ceP2K9nOFDqlrDmp8Gj7zDdvrYWZa0mWI5F9BGDjodBPKemoQxGhGE4X3SBBWAa4RHuRynJFA3KIQle700Qni/Shhct699xZQ09ZxJRmkL6LAhOglDK71+VLo2Jr7I2d4DtyverfGS/evE6xEAIKi/y6DAFLib+d7LruDMMwGuWcwecOTbp0j4KtusnWP7SCu/zTG8/y1AfeaS6OINwC5tcwg35ObIG5x+eOU/HU5nXogyE07w== yiny@Ares

Add SSH key

然后点击页面下面的添加SSH密钥这个按钮

公钥一般在C:\Users"你的用户名".ssh\id_rsa.pub里面,你需要打开隐藏的文件





A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Owner *



geekhall-laoyang ▾

/

Repository name *

Great repository names are short and memorable. Need inspiration? How about **automatic-octo-doodle?**

Description (optional)



Public

Anyone on the internet can see this repository. You choose who can commit.



Private

You choose who can see and commit to this repository.

Initialize this repository with:



owner

repository name

geekhall-laoyang / first-repo ✓

Great repository names are short and memorable. Need inspiration? How about [automatic-octo-doodle?](#)

Description (optional)

☒ **Public**
Anyone on the internet can see this repository. You choose who can commit.

☐ **Private**
You choose who can see and commit to this repository.

Initialize this repository with:
Skip this step if you're importing an existing repository.

☐ **Add a README file**
This is where you can write a long description for your project. [Learn more.](#)

Add .gitignore
Choose which files not to track from a list of templates. [Learn more.](#)

.gitignore template: None ▾

Choose a license
A license tells others what they can and can't do with your code. [Learn more.](#)

License: None ▾



Quick setup — if you've done this kind of thing before

 Set up in Desktop

or

HTTPS

SSH

`git@github.com:geekhall-laoyang/first-repo.git`



Get started by [creating a new file](#) or [uploading an existing file](#). We recommend every repository include a [README](#), [LICENSE](#), and [.gitignore](#).

...or create a new repository on the command line

```
echo "# first-repo" >> README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin git@github.com:geekhall-laoyang/first-repo.git
git push -u origin main
```



...or push an existing repository from the command line

```
git remote add origin git@github.com:geekhall-laoyang/first-repo.git
git branch -M main
git push -u origin main
```



...or import code from another repository

You can initialize this repository with code from a Subversion, Mercurial, or TFS project.





Quick setup — if you've done this kind of thing before

 Set up in Desktop or **HTTPS** **SSH** `git@github.com:geekhall-laoyang/first-repo.git` 

Get started by creating a new file or unkl... include a README, LICENSE and .gitignore.

...or create a new repository

```
echo "# first-repo" > README.md
git init
git add README.md
git commit -m "first commit"
git branch -M main
git remote add origin git@github.com:geekhall-laoyang/first-repo.git
git push -u origin main
```

`git remote add <shortname> <url>`

...or push an existing repository from the command line

```
git remote add origin git@github.com:geekhall-laoyang/first-repo.git
git branch -M main
git push -u origin main
```



```
PS E:\gitpre> git add .
PS E:\gitpre> git commit -m "remove return 0"
[master c6ebc7c] remove return 0
 1 file changed, 1 deletion(-)
PS E:\gitpre> git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 28 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 273 bytes | 273.00 KiB/s, done.
Total 3 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/Heaticy/gitpre.git
af2427b..c6ebc7c master -> master
```





 **gitpre** PrivateUnwatch 1

🔗 master ▾ 1 Branch 0 Tags

 **Heaticy** add .gitignore

 .gitignore

add .gitignore

 1.txt

add .gitignore

 a.c

add .gitignore

📖 README

Local

Codespaces

📄 Clone

HTTPS SSH GitHub CLI

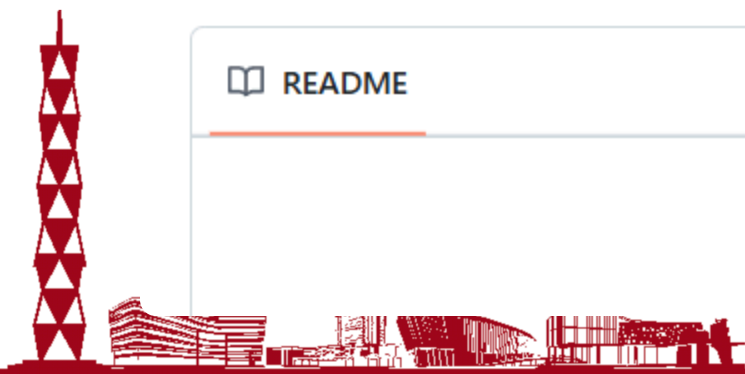
git@github.com:Heaticy/gitpre.git

📄

Use a password-protected SSH key.

🖥️ Open with GitHub Desktop

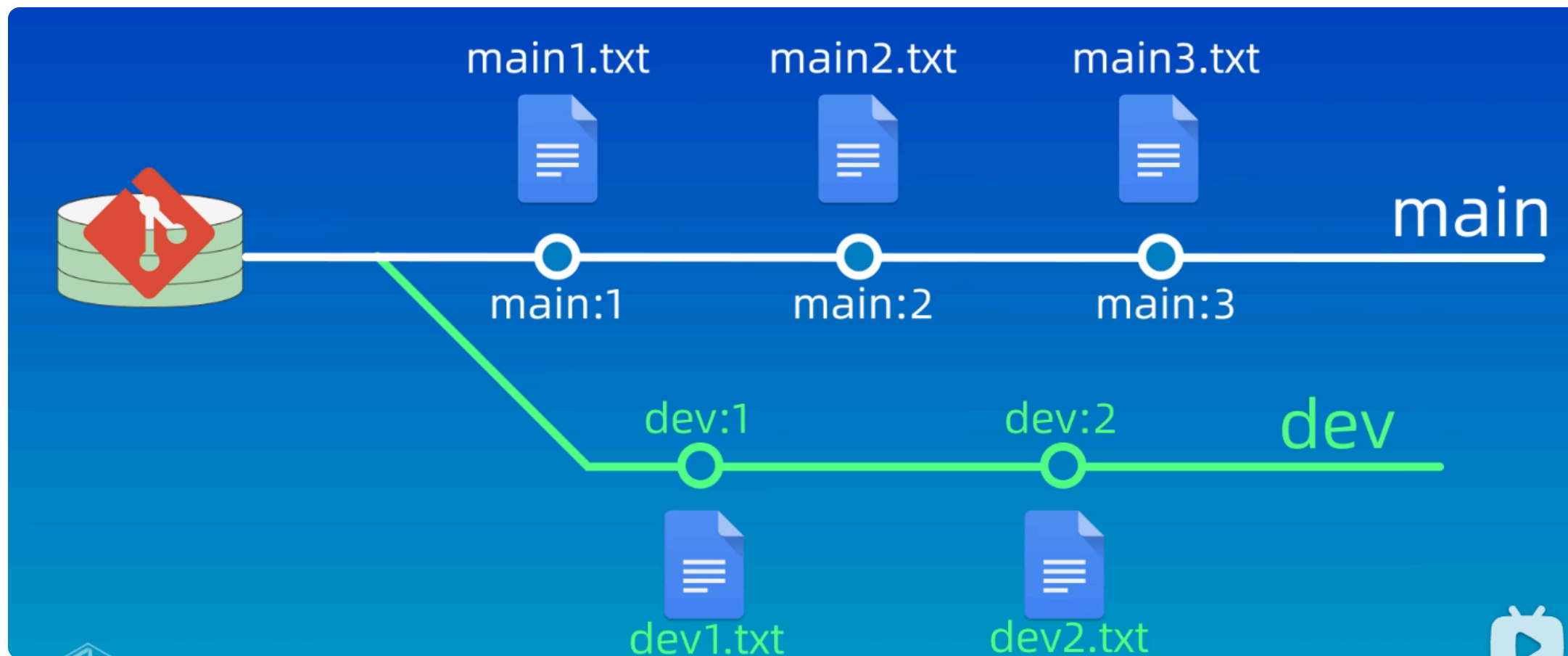
📄 Download ZIP





```
PS D:\> git clone git@github.com:Heaticy/gitpre.git
Cloning into 'gitpre'...
remote: Enumerating objects: 14, done.
remote: Counting objects: 100% (14/14), done.
remote: Compressing objects: 100% (7/7), done.
remote: Total 14 (delta 2), reused 14 (delta 2), pack-reused 0 (from 0)
Receiving objects: 100% (14/14), done.
Resolving deltas: 100% (2/2), done.
```







```
PS E:\gitpre> git branch
* main
PS E:\gitpre> git branch dev
PS E:\gitpre> git checkout dev
Switched to branch 'dev'
PS E:\gitpre> git branch
* dev
  main
  PS E:\gitpre> git add .
PS E:\gitpre> git commit -m "dev first commit"
[dev e93eddb] dev first commit
1 file changed, 6 insertions(+)
```

```
PS E:\gitpre> git push
fatal: The current branch dev has no upstream branch.
To push the current branch and set the remote as upst
```

```
git push --set-upstream origin dev
```

To have this happen automatically for branches without upstream, see 'push.autoSetupRemote' in 'git help config'.

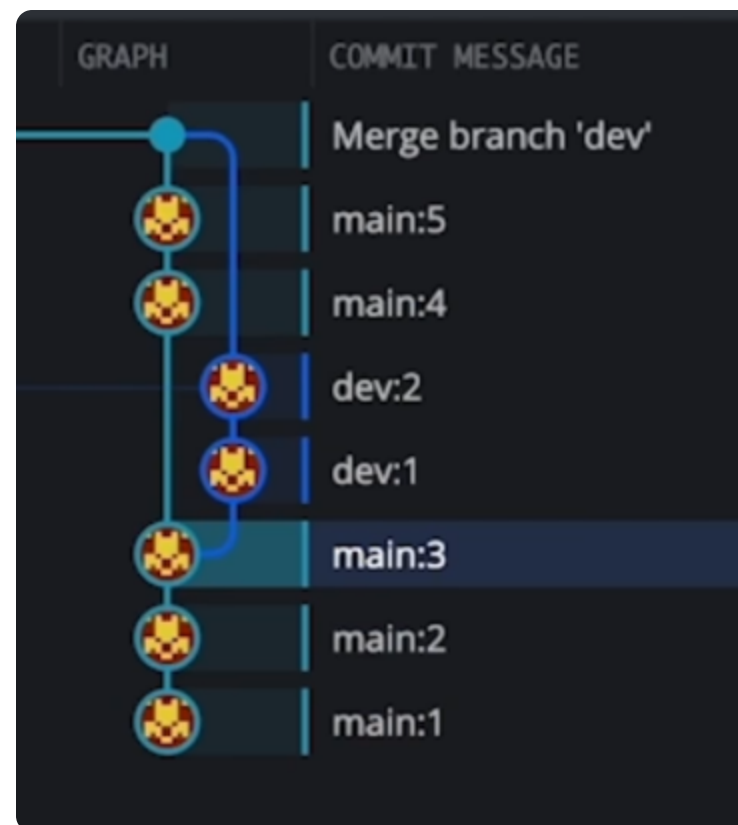
```
PS E:\gitpre> git push --set-upstream origin dev
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 28 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 337 bytes | 337.00 KiB/s
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (1/1), completed with
remote:
remote: Create a pull request for 'dev' on GitHub by
remote: https://github.com/Heaticy/gitpre/pull/new
remote:
```





- 合并

```
PS E:\gitpre> git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
PS E:\gitpre> git merge dev
Updating 76d5eb1..e93eddb
Fast-forward
 a.c | 6 ++++++
 1 file changed, 6 insertions(+)
PS E:\gitpre> git branch
  dev
* main
PS E:\gitpre> git push
```





- 合并冲突

A和B 协作

最开始main下面没有return

A加了return 0

B加了return 1

A先push, B后push

Bpush的时候则显示合并冲突

提交则需要手动修改文件去解决冲突

而使用vscode可以更好帮你去快速解决冲突





两个分支未修改同一个文件的同一处位置：Git 自动合并

两个分支修改了同一个文件的同一处位置：产生冲突

解决方法：

Step1 - 手工修改冲突文件，合并冲突内容

Step2 - 添加暂存区 `$ git add file`

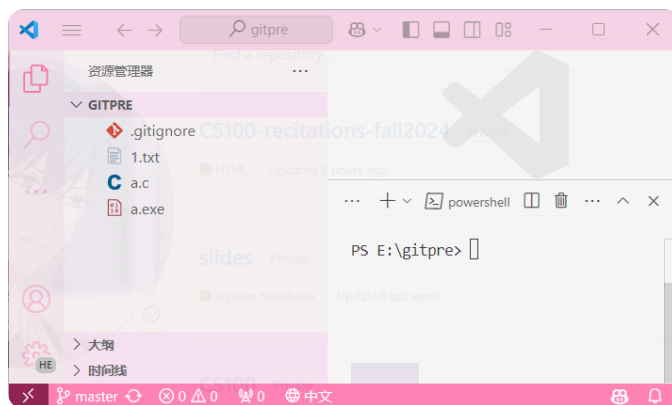
Step3 - 提交修改 `$ git commit -m "message"`

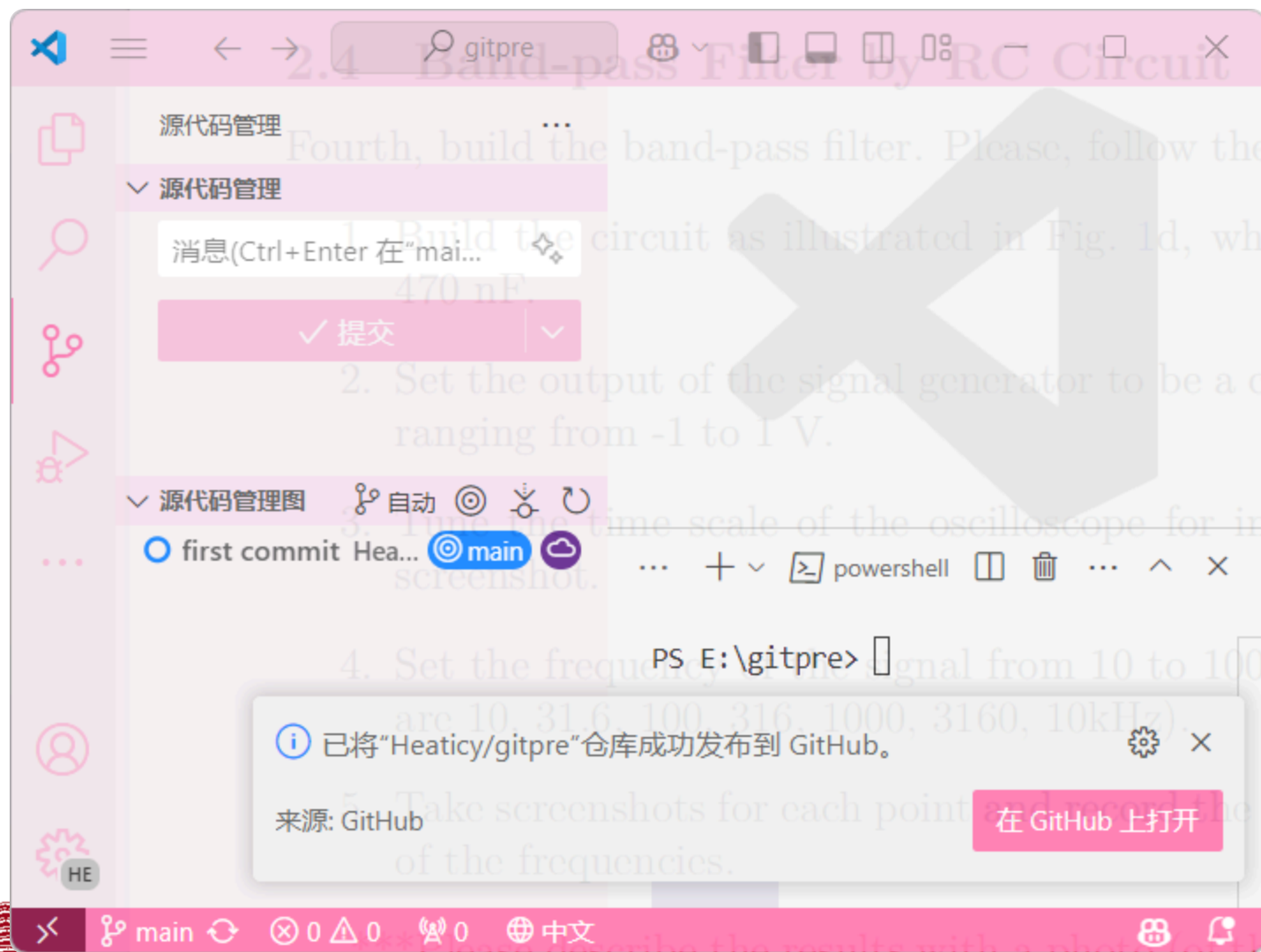
中止合并：当不想继续执行合并操作时可以使用下面的命令来中止合并过程：

`$ git merge --abort`



vscode上面的git比terminal稍微friendly一些







修改

```
C a.c M X
C a.c > main()
2
3 int main()
4 {
5     printf("Hello, World!\n");
6     return 0;
7 }
```

diff (比较)

暂存区和工作区比较

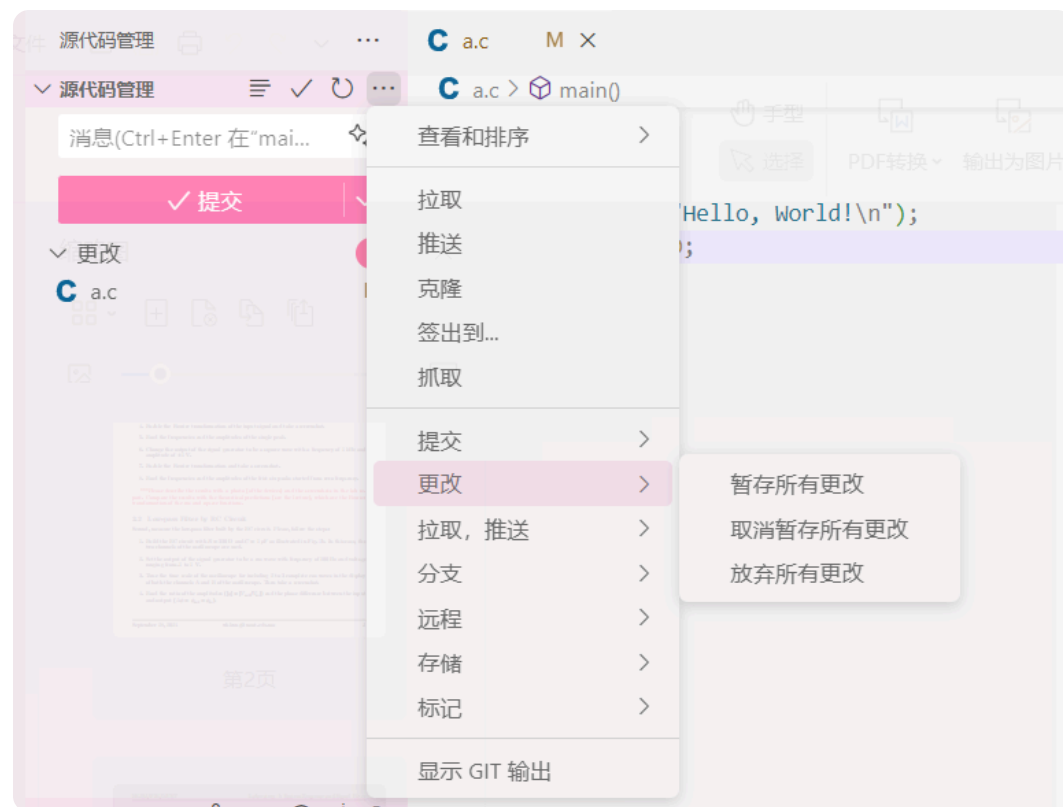
```
C a.c M X
C a.c > main()
1 #include <stdio.h>
2
3 int main()
4 {
5     printf("Hello, World!\n");
6     return 0;
7 }
```

Git 本地工作更改 - 第 1 个更改(共 1 个)

```
3 3 int main()
4 4 {
5 5     printf("Hello, World!\n");
6 6     return 0;
6 7 }
```

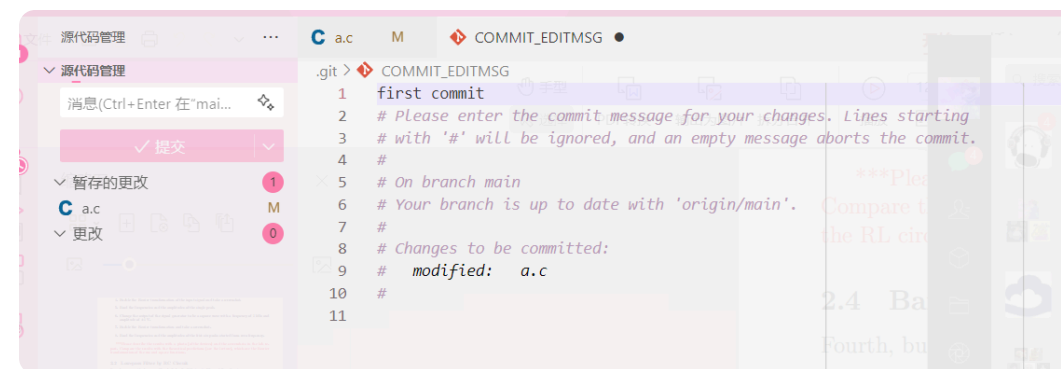


暂存

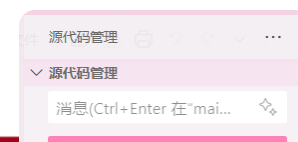


提交

在第一行输入提交信息，这个时候不是卡了，而是等待你保存这个文件，然后关闭



上传



点击同步更改即为 git push



上海科技大学
ShanghaiTech University

From Chaofan Li



立志成才报国 裕民